

Single argument `std::inserter`

- Document number: P0471R0
- Date: 2016-10-14
- Reply-to: David Sankel <dsankel@bloomberg.net>
- Audience: Library Evolution

Abstract

before	after
<pre>std::vector<std::string> vec = /* ... */; std::set<std::string> set; std::copy(vec.begin(), vec.end(), std::inserter(set, set.begin()));</pre>	<pre>std::vector<std::string> vec = /* ... */; std::set<std::string> set; std::copy(vec.begin(), vec.end(), std::inserter(set));</pre>

`std::inserter` is most commonly used for copying data from a vector to something like a set where `std::back_inserter` doesn't work. Search your code base for usages of `std::inserter` and you'll discover the "hint" argument is consistently and indiscriminately either the container's begin or end iterator. Because this "hint" is never used to hint anything, we propose an overload of the `std::inserter` function template that has a single container parameter and returns a corresponding insert iterator. This change in the standard library would simplify a common operation that is a challenge to both teach and explain.

The Chosen Iterator

The proposed single argument version of `std::inserter` must choose an iterator to pass on to the `std::insert_iterator` it constructs. There are three possibilities. Given a passed container `x`, we can

- always use `x.begin()`,
- always use `x.end()`, or
- leave it unspecified.

These options are semantically equivalent for containers like `std::set`. This choice has more important consequences on order preserving types like `std::vector`. It is the opinion of the author that the choice of least surprise is to always use `x.end()`.

Wording

If the committee deems this is worth pursuing, wording will be provided.

Conclusion

This minor change improves readability by better supporting the most common usage pattern. Minor improvements like this could go a long way towards improving C++'s ability keep simple code simple.